2026年4月7日

OshiruZine-AI

人工知能・機械学習・LLMを中心に、
技術トレンドから実用事例まで幅広く扱う技術系雑誌。

OshiruZine-AI

2026年4月7日 号

目次

1. Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス　*スタンダード（約2000字）*
2. GPT-5・Gemini 2.5・Claude 4——2026年春のフロンティアモデル徹底比較と選定ガイド　*特集（約4000字）*
3. AIエージェント基盤の本命は？ MCP・A2A・OpenAI Agents SDKが描くエージェント間連携の現在地　*スタンダード（約2000字）*
4. EU AI Act全面施行から3か月——日本企業が今すぐ対応すべきコンプライアンスチェックリスト　*ショート（500〜1000字）*

Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス

2026年に入り、AIコーディングアシスタントの勢力図は大きく塗り替わった。Anthropicの「Claude Code」とOpenAIの「Codex」は、もはや単なる補完ツールではなく、ソフトウェア開発のワークフローそのものを再定義する存在になりつつある。ターミナルに常駐し、リポジトリ全体を文脈として理解し、設計からテスト・デプロイまでを一気通貫で支援する——そんな世界がすでに現実のものとなっている。本コラムでは、2026年4月時点での最新動向を踏まえ、これらのツールを最大限に活かすためのベストプラクティスを整理する。

エージェント型開発の時代が本格到来

2025年後半から2026年にかけて、AIコーディングツールは「補完型」から「エージェント型」へと明確にシフトした。Claude Codeはターミナルネイティブのエージェントとして、ファイルの読み書き、コマンド実行、Git操作、さらには外部API呼び出しまでを自律的にこなす。一方、OpenAIが2025年にリリースしたCodex CLIも、codex-1モデルを基盤にサンドボックス環境内でコードを生成・実行・検証するエージェントループを実装している。

重要なのは、これらのツールが「指示を出したら完成品が返ってくる魔法の箱」ではないという点だ。エージェントの自律性が高まるほど、人間側の指示設計（プロンプトエンジニアリング）とガードレール設定が成果を左右する。ここからは、実践で効果が実証されているプラクティスを具体的に見ていこう。

CLAUDE.md / システムプロンプトによるコンテキスト制御

Claude Codeを使う際に最も効果的とされるのが、リポジトリのルートに配置する`CLAUDE.md`ファイルの活用だ。このファイルはClaude Codeがセッション開始時に自動で読み込むプロジェクト固有の指示書として機能する。

ベストプラクティスとして推奨されるのは以下の構成だ。

- **プロジェクトの概要とアーキテクチャ**：使用言語、フレームワーク、ディレクトリ構造の説明
- **コーディング規約**：命名規則、エラーハンドリング方針、禁止パターン
- **ビルド・テストコマンド**：`npm run test`や`cargo build`など、検証に必要なコマンド一覧
- **やってはいけないこと**：本番DBへの直接接続禁止、特定ディレクトリの変更禁止など

Codex側でも同様に、`.codex/`ディレクトリやシステムプロンプトの設定で類似のコンテキスト注入が可能だ。いずれのツールでも、「暗黙知をテキストとして明示する」ことがエージェントの精度を劇的に向上させる。

タスク分割と段階的承認のワークフロー

エージェントに大きなタスクを一括で投げるのは、2026年現在でもアンチパターンだ。Claude Codeの`/plan`モードやCodexの`suggest`（デフォルト）モードを活用し、まず計画を立案させ、人間がレビューした上で実行に移す段階的アプローチが推奨される。

具体的なワークフロー例を示す。

1. **計画フェーズ**：「この機能を実装するための手順を提案して」と指示し、変更対象ファイルと手順の一覧を出力させる
2. **レビューフェーズ**：提案内容を人間が確認し、不要な変更や設計上の懸念を指摘する

3. **実行フェーズ**：承認後、エージェントにコード生成・テスト実行まで一気に進めさせる
4. **検証フェーズ**：差分をGitで確認し、CIパイプラインを通してマージする

Claude Codeでは自律性レベルを`plan`、`autofix`、`full auto`の段階で切り替えられる。チーム開発では`autofix`（lint・テストエラーの自動修正まで許可）を基本とし、`full auto`は信頼性の高い定型タスクにのみ適用するのが安全策だ。

テスト駆動でエージェントの出力品質を担保する

AIエージェントが生成したコードの品質をどう担保するか——この問いに対する2026年時点での最良の答えは「テスト駆動開発（TDD）との組み合わせ」だ。

先にテストコードを書き（あるいはエージェントに書かせ）、そのテストをパスする実装をエージェントに生成させる。Claude Codeは`CLAUDE.md`にテストコマンドを記載しておけば、コード変更後に自動でテストを実行し、失敗した場合は自己修正ループに入る。Codexもサンドボックス内でテスト実行と修正を繰り返す仕組みを備えている。

この手法の利点は、エージェントの「もっともらしいが微妙に間違ったコード」を機械的に検出できることにある。型チェック、linter、既存のテストスイートをフル活用し、エージェントの出力を常に検証可能な状態に置くことが不可欠だ。

セキュリティとサンドボックスの意識

エージェントにシェルコマンドの実行権限を与えるということは、潜在的なリスクを伴う。Claude Codeは許可リスト方式でコマンド実行を制御でき、Codexはネットワーク分離されたサンドボックス内で動作する設計を採用している。

チーム導入時には以下を徹底すべきだ。

- **シークレット情報の分離**：`.env`ファイルや認証情報をエージェントのコンテキストに含めない
- **権限の最小化**：CI環境での実行時は読み取り専用モードを基本とする
- **監査ログの保持**：エージェントが実行したコマンドと変更内容を記録する

ツール選定と使い分けの指針

Claude CodeとCodexのどちらを選ぶかは、チームの技術スタックと開発スタイルに依存する。Claude Codeはコンテキストウィンドウの広さとCLAUDE.mdによる柔軟なカスタマイズ性に強みがあり、大規模リポジトリでのリファクタリングや複雑なマルチファイル変更に向く。Codexはサンドボックスの堅牢性とGitHub連携の完成度が高く、Issue起点の自動実装やCI統合に適している。

もちろん、両者を併用するチームも増えている。重要なのはツールへの過度な依存ではなく、人間がアーキテクチャの意思決定を握り、エージェントを「超高速で動く優秀なジュニアエンジニア」として適切にマネジメントすることだ。

まとめ——人間の設計力がAIの出力を決める

Claude CodeもCodexも、日進月歩で進化を続けている。しかし、どれだけツールが高度化しても、プロジェクトの文脈を正しく伝え、適切な粒度でタスクを分割し、出力を検証する仕組みを整えるのは人間の役割だ。最新のベストプラクティスを取り入れつつ、「AIと協働する開発者」としてのスキルセットを磨いていくことが、これからのエンジニアに求められる最も重要な投資となるだろう。

GPT-5・Gemini 2.5・Claude 4——2026年春のフロンティアモデル徹底比較と選定ガイド

2026年の春、AIフロンティアモデルの勢力図がまた大きく塗り替わろうとしている。OpenAIは2月にGPT-5を正式リリースし、Googleは3月にGemini 2.5 Proの一般提供を開始、そしてAnthropicは同じく3月末にClaude 4ファミリーを公開した。3社がほぼ同時期にフラッグシップモデルを刷新するのは前例がなく、エンジニアや研究者にとっては「どのモデルを選ぶべきか」という判断がかつてないほど難しくなっている。

本稿では、この三大フロンティアモデルを**推論性能・マルチモーダル対応・コスト・APIエコシステム**の4軸で横断比較し、ユースケース別の選定指針を提示する。ベンチマークの数字だけでは見えない実務上の差異にも踏み込んでいくので、モデル選定に悩む現場のエンジニアにとって実践的な羅針盤となれば幸いだ。

2026年春・三大モデルのスペック概観

まず、各モデルの公開情報をもとにスペックを整理しておこう。

GPT-5（OpenAI／2026年2月GA） OpenAIが約1年半ぶりにメジャーバージョンを更新。コンテキストウィンドウは標準で256Kトークン、拡張モードで最大1Mトークンに対応した。o3で培った推論チェーン技術が統合され、「考えるモード（reasoning mode）」をAPIパラメータで切り替えられる設計が特徴的だ。ネイティブなマルチモーダル入出力（テキスト・画像・音声・動画）に対応し、ツール呼び出しの精度も大幅に向上している。

Gemini 2.5 Pro（Google DeepMind／2026年3月GA） 2025年末のプレビュー期間を経て一般提供に至ったGemini 2.5 Pro は、最大2Mトークンという圧倒的なコンテキスト長を維持しつつ、「Deep Think」と呼ばれる拡張推論モードを搭載。Google検索やGoogle Workspace、BigQueryとのネイティブ連携がVertex AI上で提供される。動画理解の精度ではベンチマーク上トップクラスのスコアを記録している。

Claude 4（Anthropic／2026年3月GA） Claude 3.5世代の約1年後に登場したClaude 4ファミリーは、Claude 4 Opus・Claude 4 Sonnet・Claude 4 Haikuの3階層構成。Opusは最大512Kトークンのコンテキストに対応し、長文の構造理解と指示追従性に定評がある。「拡張思考（extended thinking）」がOpusおよびSonnetで利用可能となり、数学・コーディング・複雑な分析タスクでの推論精度が飛躍的に改善された。安全性フレームワークの透明性もさらに強化されている。

推論性能——ベンチマークと実務の両面から

ベンチマーク上の比較

2026年3月時点で公開されている主要ベンチマーク（MMLU-Pro、GPQA Diamond、HumanEval-Plus、MATH-500、ARC-AGI-2）のスコアを総合すると、概ね以下のような傾向が見て取れる。

- **数学・科学推論**: GPT-5（reasoningモード）とClaude 4 Opus（extended thinking）がほぼ拮抗し、GPQA DiamondおよびMATH-500で80%台後半を記録。Gemini 2.5 Pro Deep Thinkもこれに迫るが、数学の超難問カテゴリではやや差がつく場面がある。
- **コーディング**: HumanEval-Plusのpass@1ではClaude 4 Opusが最高スコアを出す一方、SWE-bench Verifiedなど実世界のソフトウェアエンジニアリングタスクではGPT-5が安定して高いパフォーマンスを示す。Gemini 2.5

Proはコード生成よりもコードレビュー・バグ検出に強みがあるとの報告が多い。

- **汎用知識・推論**: MMLU-Proでは三者ともに90%前後で横並びとなり、もはやこのベンチマークだけでは差別化が難しい水準に達している。

実務での体感差

ベンチマークでは見えにくいが、実務で顕著な差が出るのは**指示追従性（instruction following）**と**出力の安定性（consistency）**だ。

Claude 4 Opusは長い系統的な指示を与えた際の逸脱が少なく、企業向けドキュメント生成やコンプライアンスチェックのような「指示通りのフォーマットで出力してほしい」用途で非常に高い信頼性を発揮する。GPT-5はクリエイティブタスクでの柔軟性に優れ、指示の行間を読んで補完する力が強い半面、厳密なフォーマット準拠ではやや"遊び"が出やすい。Gemini 2.5 Proはグラウンディング（Google検索連携）を活かした事実確認付き回答に強く、最新情報を参照する必要があるユースケースでは頭一つ抜けている。

マルチモーダル対応——テキストだけでは語れない時代

2026年春時点で、三大モデルすべてがテキスト・画像・音声の入力に対応している。ただし**出力モダリティ**と**動画理解**で差が生まれている。

画像生成・編集: GPT-5はDALL·E 4エンジンを統合し、会話の中でシームレスに画像を生成・編集できる。Geminiも画像生成に対応しているが、Imagen 3ベースの別APIを経由する構成で、プロンプト内での統合度はGPT-5に軍配が上がる。Claude 4は画像入力の解析精度は極めて高いものの、画像生成機能は提供していない。

動画理解: Gemini 2.5 Proが最も成熟している。YouTube動画をURLで渡して要約・分析させるユースケースはVertex AIのキラー機能の一つだ。GPT-5も動画

入力に対応しているが、処理可能な長さと精度ではGeminiが優位。Claude 4は動画対応を段階的に進めているが、2026年4月時点では限定ベータに留まる。

音声: GPT-5のリアルタイムAPIは低レイテンシの音声対話を実現し、カスタマーサポートや音声エージェント構築の分野で圧倒的な採用実績がある。Gemini 2.5 ProもLive APIで双方向音声に対応し、遅延・音質ともに改善が著しい。Claude 4は音声入力をテキスト変換して処理する方式をとっており、リアルタイム音声対話には未対応だ。

まとめると、マルチモーダルの総合力ではGPT-5がリード、動画特化ならGemini、テキスト＋画像解析の精度重視ならClaude 4、という棲み分けが明確になっている。

コスト——トークン単価だけでは判断できない

コスト比較は単純なトークン単価だけでなく、**スループット・キャッシュ機構・バッチ処理割引**まで含めて見る必要がある。2026年4月時点のAPI公式価格（入力/出力、100万トークンあたり、USD）を概算で示す。

モデル	入力	出力	備考
GPT-5	$12	$40	reasoningモード利用時はthinkingトークン分が加算
GPT-5 Mini	$2	$8	軽量タスク向け
Gemini 2.5 Pro	$7	$28	128K超のコンテキストは割増料金
Gemini 2.5 Flash	$0.50	$2	コスト最適化用途
Claude 4 Opus	$15	$75	extended thinking利用時はthinkingトークン加算

モデル	入力	出力	備考
Claude 4 Sonnet	$4	$16	コスパのバランス型
Claude 4 Haiku	$0.80	$3.20	高速・低コスト

注目すべきは**プロンプトキャッシュ**の存在だ。OpenAIとAnthropicはともに、同一プレフィックスの再利用で入力トークンコストを大幅に削減できるキャッシュ機構を提供している。大量ドキュメントに対して同一のシステムプロンプトを使い回すRAG構成では、キャッシュのヒット率次第で実質コストが公示価格の半分以下になることも珍しくない。Geminiも同様のContext Caching機能をVertex AIで提供しており、長時間のコンテキスト保持が可能だ。

バッチAPIによる非同期処理割引も見逃せない。OpenAIのBatch APIは50%割引、Anthropicも同様の仕組みを導入しており、リアルタイム性が不要な分析パイプラインではこれを活用しない手はない。

実務上のコスト最適化で最も効くのは、**タスクの難易度に応じたモデルのルーティング**だ。簡易な分類・抽出にはHaikuやFlash、標準的な生成にはSonnet、高度な推論が必要な場面だけOpusやGPT-5を呼ぶ——こうしたカスケード構成を自動化するミドルウェア（LiteLLM、OpenRouter、独自ルーターなど）の導入が2026年のベストプラクティスになりつつある。

APIエコシステム——モデル単体では戦えない

モデルの性能が拮抗する時代、差別化の焦点は**エコシステムの豊かさ**に移っている。

OpenAI: Assistants APIの成熟度が高く、ファイル検索・コードインタープリター・関数呼び出しを一つのスレッドで統合管理できる。GPT-5のリリースと同時にResponses APIが新たな標準エンドポイントとなり、ツール連携の柔軟性がさ

らに向上した。Azure OpenAI Serviceを通じたエンタープライズ向けデプロイメントも盤石で、SOC 2やHIPAAなどのコンプライアンス要件を満たしやすい。

Google: Vertex AIを軸に、BigQuery・Cloud Storage・Google Workspaceとのネイティブ統合が最大の強み。Grounding with Google Searchによって回答に引用元URLを付与できる機能は、企業のファクトチェックワークフローに直結する。Agentspace（旧NotebookLM Enterprise）やAgent Development Kit（ADK）など、エージェント構築基盤への投資も積極的だ。

Anthropic: Model Context Protocol（MCP）がオープンスタンダードとして業界で急速に採用されており、外部ツール・データソースとの接続を標準化する動きの中心にいる。Claude 4ではcomputer use（PC操作エージェント）が正式機能となり、ブラウザ操作やデスクトップ自動化のユースケースで独自のポジションを築いている。API Consoleのプロジェクト管理機能やワークベンチも着実に改善が進んでいる。

ユースケース別・選定ガイド

ここまでの比較を踏まえ、代表的なユースケースごとの推奨モデルを整理する。

1. RAGベースの社内ナレッジ検索 → Claude 4 Sonnet または **Gemini 2.5 Pro**。長文コンテキストの正確な引用と指示追従性が重要。コスト効率を考慮するとSonnetがバランスに優れるが、最新のWeb情報も参照したい場合はGeminiのグラウンディング機能が有力。

2. コード生成・リファクタリング支援 → Claude 4 Opus（高精度）/ **GPT-5**（幅広い言語・フレームワーク対応）。SWE-benchの実績と、IDE連携プラグインの充実度で選ぶとよい。GitHub Copilotとの統合を重視するならGPT-5のエコシステムが一歩リードしている。

3. マルチモーダルコンテンツ生成 → GPT-5。画像生成・音声対話を含む統合的なコンテンツパイプラインを一つのAPIで完結させたい場合は現時点で最適解。

4. 大規模データの分析・要約 → Gemini 2.5 Pro。2Mトークンのコンテキストウィンドウを活かし、大量のCSVやログファイルを一括で投入して分析させる用途ではGeminiの右に出るモデルはない。

5. エージェント・自動化ワークフロー → Claude 4 Opus + MCP / GPT-5 + Responses API。ツール連携の安定性と、長時間のエージェントループでの一貫性がカギ。AnthropicのMCPエコシステムは接続先の拡大が著しく、Slackやデータベースとの連携プラグインが充実している。GPT-5のResponses APIも組み込みツールの豊富さで勝負できる。

6. コスト最優先の大量バッチ処理 → Gemini 2.5 Flash / Claude 4 Haiku。品質要件が許す範囲で軽量モデルを使い、バッチAPIで非同期処理するのが最もコスト効率が高い。

選定で本当に大事な3つの判断軸

最後に、モデル選定において最も重要な判断軸を3つに絞って提示したい。

① レイテンシ要件を明確にする。リアルタイム対話が必要なのか、バッチ処理でよいのか。これだけでモデルとプランの選択肢が大きく絞り込まれる。

② ロックインのリスクを評価する。特定プロバイダーのエコシステムに深く依存するほど移行コストは上がる。抽象化レイヤー（LiteLLM、LangChainのプロバイダー抽象化など）を挟み、モデル切り替えの柔軟性を確保することを推奨する。

③ 「今」のベンチマークだけで決めない。フロンティアモデルの性能差は数ヶ月単位で逆転しうる。重要なのは、自社のユースケースに対して**独自の評価セット（eval）を構築し、新モデルが出るたびに定量的に再評価する仕組み**を持つことだ。

おわりに

GPT-5・Gemini 2.5 Pro・Claude 4——2026年春のフロンティアモデルは、いずれも単体で「圧倒的な勝者」とは言い難い。むしろ、それぞれが明確な強みを持ち、ユースケースによって最適解が異なる「三国志」の様相を呈している。

だからこそ、エンジニアに求められるのは「最強のモデルを一つ選ぶ」ことではなく、**タスクに応じて最適なモデルをルーティングできるアーキテクチャを設計すること**だ。モデルは入れ替わる。エコシステムも変わる。しかし、自社の課題を正しく分解し、適切な評価基準で選び続ける力——それこそが、AI活用の真の競争優位になる。

次の四半期にはまた新たなモデルが登場するだろう。そのとき、本稿で示した判断軸が皆さんの意思決定を

AIエージェント基盤の本命は？ MCP・A2A・OpenAI Agents SDKが描くエージェント間連携の現在地

「エージェントを作る」フェーズから「エージェント同士をつなぐ」フェーズへ——2025年後半から2026年初頭にかけて、AIエージェント開発の重心は明確にシフトした。単体のLLMに適切なプロンプトを与えるだけでは到底こなせない複雑なタスクを、複数の専門エージェントが協調して遂行する「マルチエージェントアーキテクチャ」が、もはやリサーチの題材ではなく本番実装の選択肢になりつつある。しかし、その接続層を担う仕様は複数が並立し、エンジニアの頭を悩ませているのも事実だ。本コラムでは、MCP（Model Context Protocol）、

Google A2A（Agent-to-Agent）プロトコル、そしてOpenAI Agents SDKの三つを軸に、それぞれの設計思想と実務上の使い分けを整理する。

MCPが解決した「コンテキスト供給」という課題

Anthropicが2024年末にオープンソースとして公開し、2025年後半にかけて急速にエコシステムを拡大したMCPは、その名の通り「モデルにコンテキストを渡す方法」を標準化するプロトコルだ。従来、エージェントが外部ツールやデータソースにアクセスするには、開発者がAPIごとにアダプタを書く必要があった。MCPはここにUSB-Cのような統一インターフェースを提供する。MCPサーバーを実装すれば、あらゆるMCP対応クライアント（Claude、Cursor、自社エージェントなど）から同じ手順でツールを呼び出せる。

2026年3月時点で、GitHub、Slack、Notion、各種RDB、クラウドストレージなど主要サービスの公式・コミュニティ製MCPサーバーは数百を超え、npmやPyPIからワンコマンドで導入できるものも多い。2025年末に追加されたStreamable HTTPトランスポートにより、ローカルプロセス間通信（stdio）だけでなくリモート環境へのデプロイも現実的になった。さらに2026年初頭に仕様へ組み込まれた認証・認可フレームワーク（OAuth 2.1ベース）は、エンタープライズ導入における最大のブロッカーを取り除きつつある。

ただし、MCPの守備範囲はあくまで「エージェント—ツール間」のコンテキスト供給であり、「エージェント—エージェント間」の対話やタスク委譲を直接扱うレイヤーではない点には注意が必要だ。

A2Aが埋める「エージェント間交渉」の空白

Googleが2025年4月に発表し、同年後半にv0.2へアップデートしたA2A（Agent-to-Agent）プロトコルは、まさにMCPが扱わない領域を補完する。A2Aの中心概念は「Agent Card」「Task」「Message/Artifact」の三つだ。Agent Cardは各エージェントが自らの能力をJSON形式で公開するディスカバリ機構であり、タスクを委譲したいエージェントはまず相手のAgent Cardを取得

して「何ができるか」を確認する。その上で、構造化されたTaskオブジェクトを介して依頼・進捗確認・成果物受け取りを行う。

設計上の最大の特徴は「不透明性（Opaque Execution）」を許容している点にある。委譲先エージェントが内部でどのモデルを使い、どんなツールチェーンを走らせているかを呼び出し側は知る必要がない。これはエンタープライズにおいて、部門ごとに異なるベンダーのエージェントを導入しつつ全体をオーケストレーションするユースケースに極めて合致する。2026年2月にはAtlassian、Salesforce、SAP、Box など50社超がA2Aパートナーとして参加しており、実装例も増えてきた。

ただし、A2AはHTTPベースのリクエスト/レスポンス＋SSEストリーミングという比較的シンプルなトランスポートに依存しており、リアルタイム性の高い双方向対話にはまだ課題が残る。仕様自体もまだ成熟途上であり、プロダクション投入にはラッパー層での補強が求められる場面がある。

OpenAI Agents SDKのアプローチ——「フレームワーク内完結」の実利

一方、OpenAIが2025年3月にリリースし、その後も継続的にアップデートを重ねているAgents SDKは、プロトコルというよりも「マルチエージェント構築フレームワーク」に近い。Agent、Handoff、Guardrail、Tracingといったプリミティブを数十行のPythonコードで組み合わせ、エージェント間のタスク委譲（Handoff）やセーフティレイヤーを宣言的に記述できる。

最大の利点はそのプラグマティズムだ。MCPツール連携がビルトインでサポートされており、MCPサーバーをtool定義にそのまま組み込める。つまり「外部ツールとのI/FはMCP、エージェント間のオーケストレーションはAgents SDK」という二層構成を、追加ライブラリなしに実現できる。2026年に入ってからはストリーミングイベントの粒度が細かくなり、UIへの中間状態反映も容易になった。

ただし、現時点ではOpenAIモデルへの依存度が高い点は無視できない。オープンモデルへの対応も進んでいるとはいえ、ベンダーニュートラルな基盤を志向する場合には設計判断が必要だ。

三者の関係は「競合」ではなく「レイヤー分担」

ここまで読んで気づかれた方も多いだろうが、MCP・A2A・Agents SDKは同じ課題を異なる角度から解いている。整理するとこうなる。

- **MCP** → エージェント—ツール間のコンテキスト供給レイヤー
- **A2A** → エージェント—エージェント間のディスカバリ・タスク委譲レイヤー
- **Agents SDK** → 単一開発者／チーム内でのマルチエージェントオーケストレーションフレームワーク

実務上の推奨パターンとしては、まずMCPで外部ツール統合を標準化し、チーム内部のエージェント連携にはAgents SDK（またはLangGraphやCrewAIなどの代替フレームワーク）を使い、組織横断・ベンダー横断のエージェント連携が必要になった段階でA2Aを導入する、というグラデーションが現実的だ。

マルチエージェント時代のエンジニアに求められる視点

標準化が進むことは歓迎すべきだが、仕様の乱立はかつてのコンテナオーケストレーション戦争を思い起こさせもする。重要なのは「どれが勝つか」を予想するゲームに時間を費やすことではなく、各レイヤーの責務を理解し、自分たちのアーキテクチャの中で適切に組み合わせることだ。MCPのツールサーバーは今日から書ける。A2AのAgent Cardを設計することで、自チームのエージェントの能力定義が驚くほど明確になる。手を動かした先にこそ、マルチエージェント時代の地図が見えてくるはずだ。

EU AI Act全面施行から3か月——日本企業が今すぐ対応すべきコンプライアンスチェックリスト

「うちはEU域内に拠点がないから関係ない」——その認識は、もう通用しない。2026年2月2日にEU AI Actの高リスクAI規制が全面施行され、約3か月が経過した今、欧州委員会は域外適用の執行体制を着々と整備している。EU市場にサービスを提供する、あるいはEU域内のユーザーに影響を及ぼすAIシステムを開発・運用するすべての企業が対象だ。日本企業も例外ではない。

何が変わったのか——高リスクAI規制の要点

EU AI Actはリスクベースの4段階分類を採用し、高リスク（High-Risk）に該当するAIシステムに対して最も厳格な義務を課す。具体的には、採用・人事評価、信用スコアリング、法執行支援、重要インフラ制御などに用いられるAIが該当する。2月の全面施行により、これらのシステムには**適合性評価の完了とCEマーキングの取得**が市場投入の前提条件となった。違反時の制裁金は最大3,500万ユーロまたは全世界売上高の7%のいずれか高い方であり、リスクは無視できない規模だ。

今すぐ着手すべきコンプライアンスチェックリスト

グローバル展開するプロダクトチームが最低限押さえるべき項目を、技術面と組織面に分けて整理した。

技術的対応

- [ ] **リスク分類の確定**：自社AIシステムがAnnex IIIの高リスクカテゴリに該当するか判定を完了しているか

- [ ] **リスク管理システムの実装**：開発〜運用の全ライフサイクルを通じた継続的リスク評価プロセスが稼働しているか
- [ ] **データガバナンス**：学習データの品質基準策定、バイアス検出・緩和パイプラインが整備されているか
- [ ] **技術文書の整備**：システム設計、学習手法、評価指標、性能限界を記載した文書がAct要件に準拠しているか
- [ ] **ログ記録と追跡可能性**：推論ログの自動記録機能が実装され、最低6か月間の保持が確保されているか
- [ ] **人間による監視（Human Oversight）**：高リスク判定を人間がオーバーライドできるインターフェースが設計されているか

組織的対応

- [ ] **AI統括責任者の指名**：コンプライアンスを統括する役職・チームが社内に設置されているか
- [ ] **EU域内の代理人（Authorised Representative）**：域外企業として登録代理人を任命済みか
- [ ] **適合性評価の計画策定**：自己適合評価で足りるか、第三者機関の関与が必要か判断を終えているか
- [ ] **インシデント報告体制**：重大な事故・権利侵害発生時に当局へ報告するフローが整っているか
- [ ] **従業員リテラシー教育**：AI Actの概要と自社への影響について、開発者・PM・経営層への研修を実施したか

まず一歩目を踏み出す

すべてを一度に完璧にする必要はない。しかし、**リスク分類の確定**と**技術文書の棚卸し**だけは今日にでも着手してほしい。この2つが定まらなければ、後続のすべての対応が宙に浮く。日本政府もAI事業者ガイドライン改訂を通じてEU規制との整合を模索している今こそ、グローバル基準への適応を"コスト"ではなく"競

争力"として捉え直す好機だ。3か月後の次の執行レビューで慌てないために、チェックリストを片手に、最初の一歩を踏み出そう。